Braeden Venne

Lab 7: Image Steganography and LSB Forensics

Part 0 – Setup and evidence inventory

```
# Uncomment if needed, then run once:
# %pip install pillow stepic

from pathlib import Path
from PIL import Image
import hashlib

ASSET_DIR = Path("Lab7")
LOGO = ASSET_DIR / "compay_logo.png"
PROFILE = ASSET_DIR / "profile.png"
PYTHON_IMG = ASSET_DIR / "python.png"
OUTPUT_DIR = Path("Lab7_outputs")
OUTPUT_DIR.mkdir(exist_ok=True)

for path in [LOGO, PROFILE, PYTHON_IMG]:
    if not path.exists():
        raise FileNotFoundError(f"Missing {path}. Check where you extracted Lab7.zip")
    with Image.open(path) as image:
        print(path.name, image.size, image.mode, path.stat().st_size, "bytes")
```

[1] ✓ 0.0s

```
compay_logo.png (1024, 1024) RGBA 1029126 bytes
profile.png (1408, 768) RGBA 2424849 bytes
python.png (100, 100) RGBA 9427 bytes
```

What does RGBA mean? Why might a forensic analyst calculate a file's SHA-256 hash?

RGBA stands for Red, Green, Blue, and Alpha channels, each of these channels are stored as 8-bit integers. A forensic analyst may calculate a file's SHA-256 hash to establish the chain of custody. Additionally, calculating a file's SHA-256 allows an analyst to verify a file's integrity, proving whether file bytes have been modified after acquisition.

```
def sha256_file(path):
    """Calculate the SHA-256 digest of a file's exact bytes."""
    with open(path, "rb") as file:
        return hashlib.sha256(file.read()).hexdigest()

# Save baseline hashes now; we will use these values again in Part 5.
original_logo_hash = sha256_file(LOGO)
original_profile_hash = sha256_file(PROFILE)

print("Original logo SHA-256:   ", original_logo_hash)
print("Original profile SHA-256:", original_profile_hash)
```

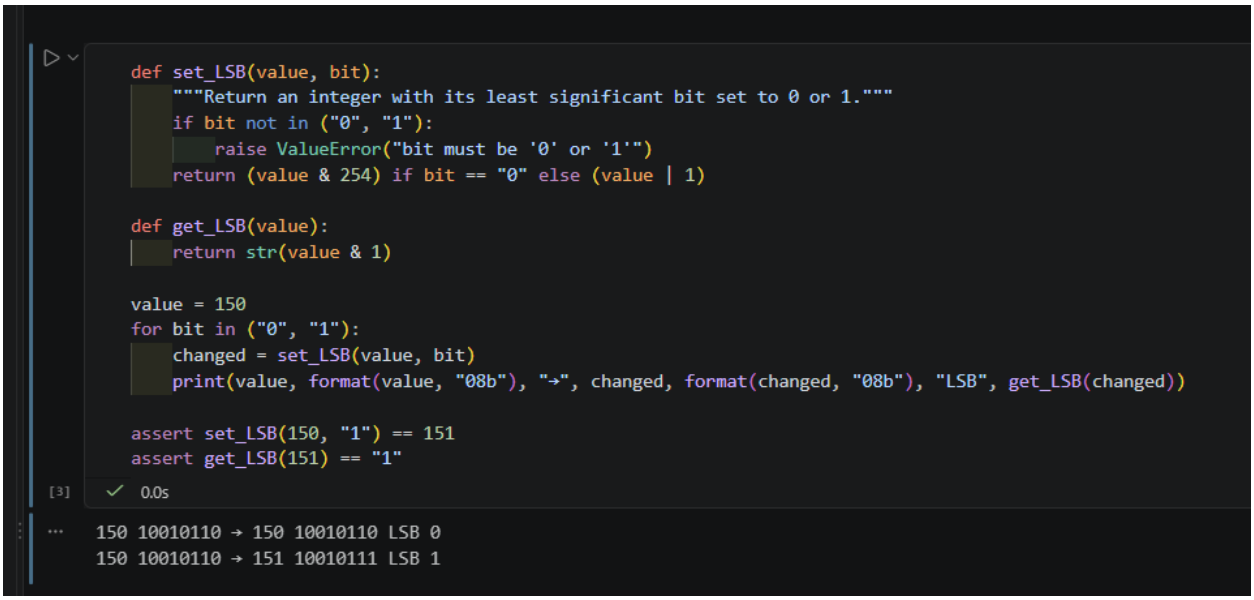
[2] ✓ 0.0s

```
Original logo SHA-256:    64303ae8e227a463b9d1c9c27359d228bf0874c1284535f6f80f3e0802a6a6e0
Original profile SHA-256: 292d9a5bcf8b10017abc1e633a86ae0ddbac4e5c2156e84d6dbf5307b3df5850
```

This shows a baseline SHA-256 hashes for the original logo and profile images, establishing a cryptographic reference for file integrity checks later in the lab.

Braeden Venne

Part 1 - Binary and LSB fundamentals



```
def set_LSB(value, bit):
    """Return an integer with its least significant bit set to 0 or 1."""
    if bit not in ("0", "1"):
        raise ValueError("bit must be '0' or '1'")
    return (value & 254) if bit == "0" else (value | 1)

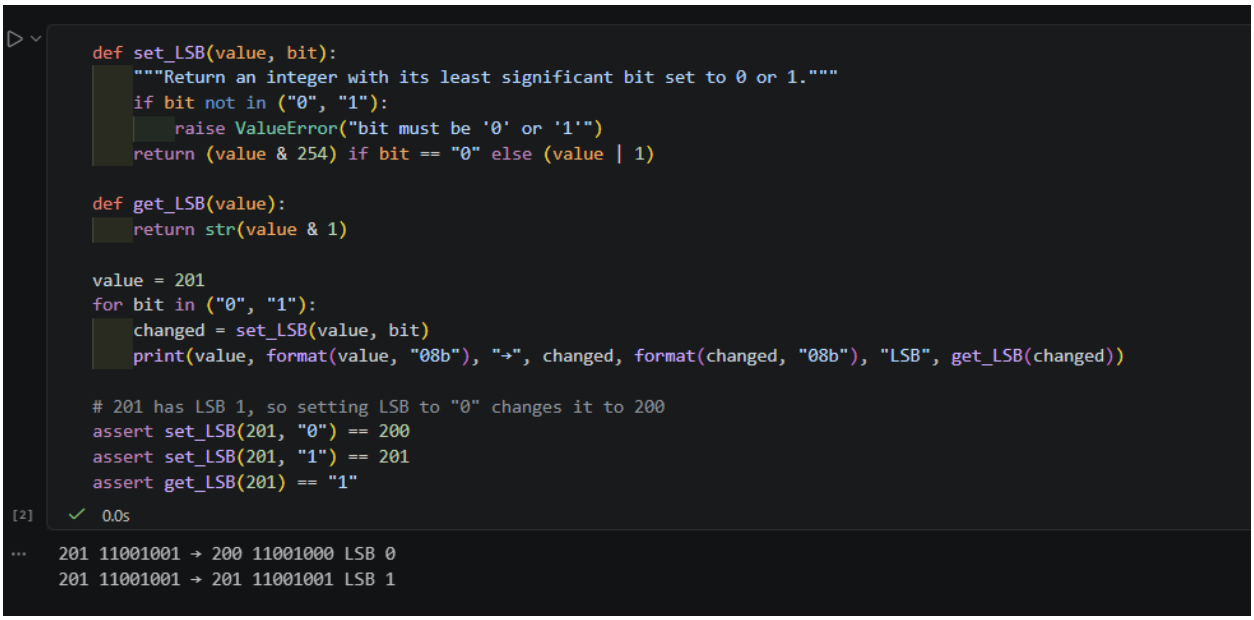
def get_LSB(value):
    return str(value & 1)

value = 150
for bit in ("0", "1"):
    changed = set_LSB(value, bit)
    print(value, format(value, "08b"), "→", changed, format(changed, "08b"), "LSB", get_LSB(changed))

assert set_LSB(150, "1") == 151
assert get_LSB(151) == "1"
```

[3] ✓ 0.0s

```
150 10010110 → 150 10010110 LSB 0
150 10010110 → 151 10010111 LSB 1
```



```
def set_LSB(value, bit):
    """Return an integer with its least significant bit set to 0 or 1."""
    if bit not in ("0", "1"):
        raise ValueError("bit must be '0' or '1'")
    return (value & 254) if bit == "0" else (value | 1)

def get_LSB(value):
    return str(value & 1)

value = 201
for bit in ("0", "1"):
    changed = set_LSB(value, bit)
    print(value, format(value, "08b"), "→", changed, format(changed, "08b"), "LSB", get_LSB(changed))

# 201 has LSB 1, so setting LSB to "0" changes it to 200
assert set_LSB(201, "0") == 200
assert set_LSB(201, "1") == 201
assert get_LSB(201) == "1"
```

[2] ✓ 0.0s

```
201 11001001 → 200 11001000 LSB 0
201 11001001 → 201 11001001 LSB 1
```

This demonstrates how modifying the least significant bit of integer values changes pixel bytes by at most 1.

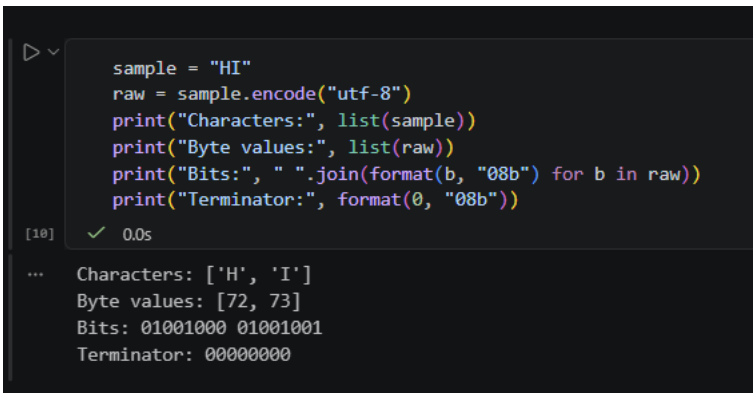
Explain why `set_LSB(201, "1")` does not change the value.

Because 201 in binary ends in 1, its least significant bit is already 1. Forcing it to 1 leaves the bit unchanged, so the value stays 201

Braeden Venne

What is the maximum numerical difference between an original channel and its LSB-modified value?

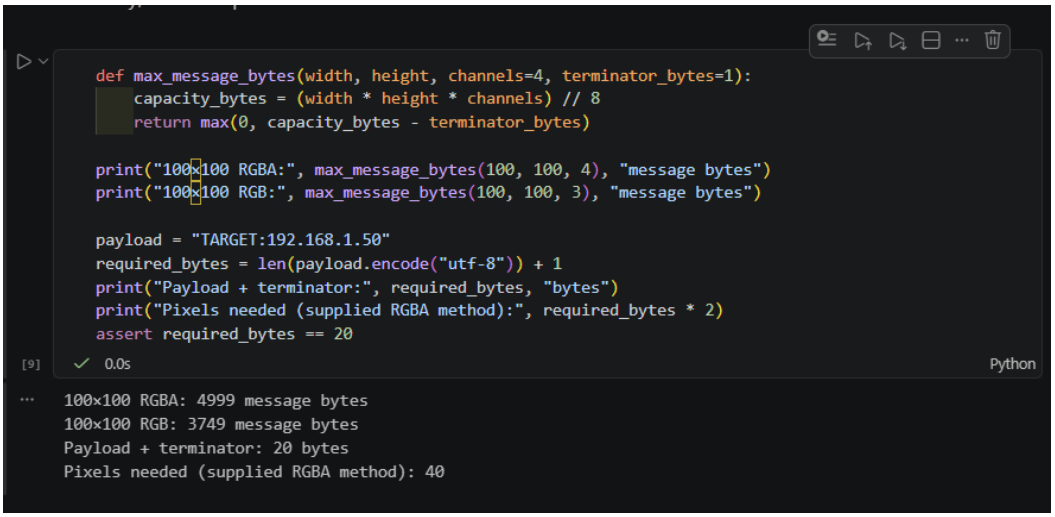
The maximum numerical difference between an original channel and its LSB-modified value is 1. Since LSB steganography only alters the least significant bit, the value can only increase or decrease by at most 1.



This shows how the string “HI” is broken down into UTF-8 byte values (72 and 73), converted into an 8-bit binary representation, and appended with a null terminator byte (00000000) to mark the end of the payload

Braeden Venne

Part 2 – Calculate Capacity Before Writing

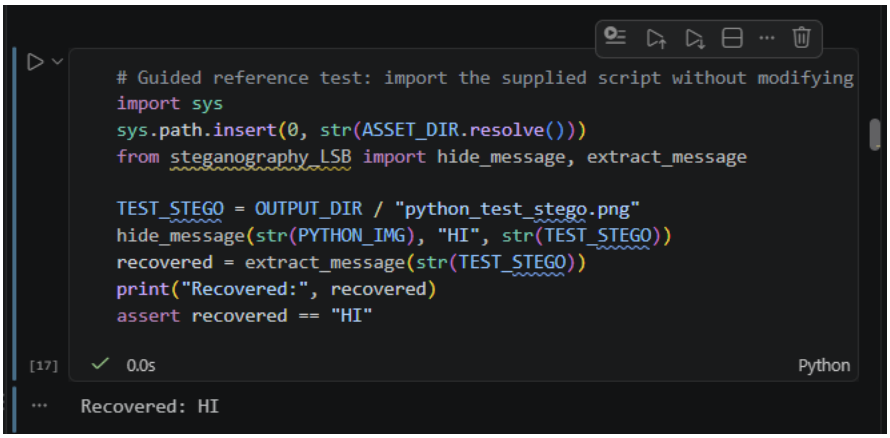


How many message bytes fit in a 100 × 100 RGBA image using this exact method? How would an RGB-only, one-LSB-per-channel method differ?

- Using this method, a 100x100 RGBA image fits 4,999 message bytes across all four channels. In comparison, an RGB-only method using one LSB per channel fits 3,749 message bytes, since it drops the alpha channel and only utilizes three channels per pixel.

Braeden Venne

Part 3 – Manual LSB Encoding and Extraction



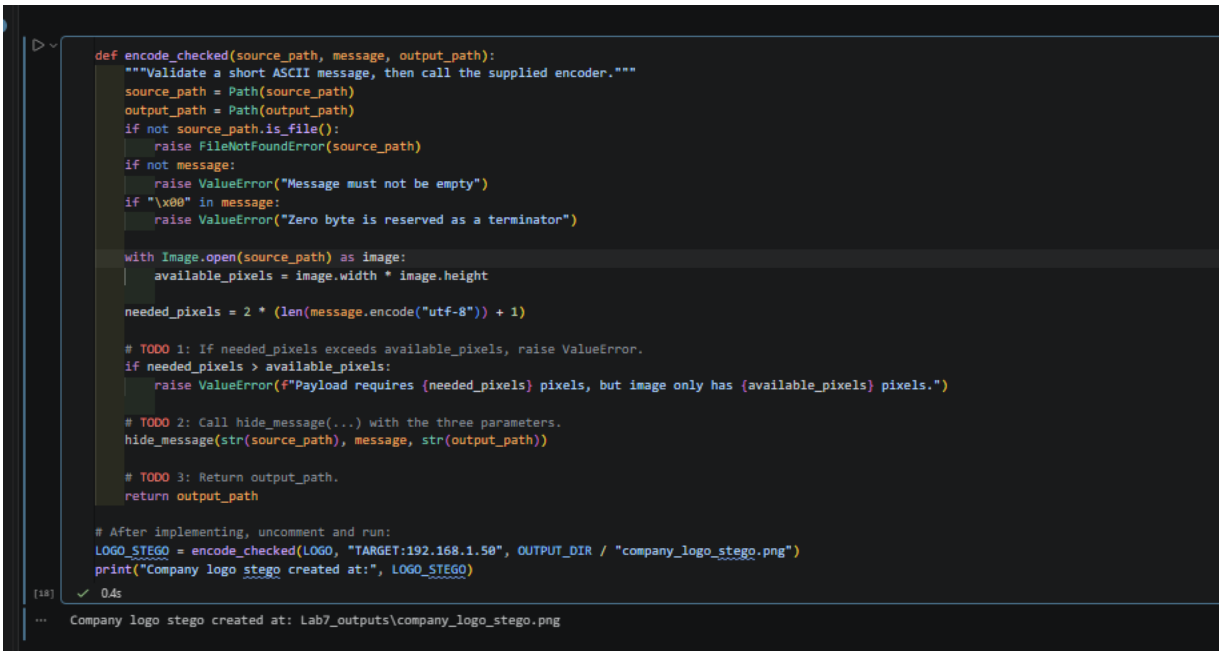
This demonstrates that the reference script successfully hid and extracted the test string “HI”, verifying that the baseline LSB implementation works as expected before building the custom encoder and decoder.

Task 3A – Inspect the Reference Code

1. Why does the reference code `call .convert("RGBA")?`
    a. It standardizes all the input carrier images to a RGBA format, ensuring uniform pixel indexing across channels regardless of whether the original image was RGB, greyscale, or indexed.
2. Why are two pixels used per byte?
    a. Each RGBA pixel has 4 channels, meaning they provide 4 LSBs. Since 1 byte requires 8 bits, two pixels are needed to store one full character byte.
3. What does `message += chr(0)` accomplish?
    a. It appends a null terminator byte to the message, telling the decoder where the message ends.
4. What happens if the payload exceeds image capacity? How could your code detect this **before** editing any pixels?
    a. If the payload exceeds image capacity, index-out-of-range errors will occur, or the image data will be corrupted. You can detect before editing any pixels by using the following code snippet:
        - `2 * (len(message. Encode("utf-8")) + 1) > width * height`
5. Why is PNG preferable to JPEG for preserving exact LSB values?
    a. PNG uses lossless compressing, preserving the exact bit values. JPEG on the other hand uses lossy compression, which alters pixel values and destroys hidden LSB data.

Braeden Venne

Task 3B – Implement a Validated Encoder



```
def encode_checked(source_path, message, output_path):
    """Validate a short ASCII message, then call the supplied encoder."""
    source_path = Path(source_path)
    output_path = Path(output_path)
    if not source_path.is_file():
        raise FileNotFoundError(source_path)
    if not message:
        raise ValueError("Message must not be empty")
    if "\x00" in message:
        raise ValueError("Zero byte is reserved as a terminator")

    with Image.open(source_path) as image:
        available_pixels = image.width * image.height

    needed_pixels = 2 * (len(message.encode("utf-8")) + 1)

    # TODO 1: If needed_pixels exceeds available_pixels, raise ValueError.
    if needed_pixels > available_pixels:
        raise ValueError(f"Payload requires {needed_pixels} pixels, but image only has {available_pixels} pixels.")

    # TODO 2: Call hide_message(...) with the three parameters.
    hide_message(str(source_path), message, str(output_path))

    # TODO 3: Return output_path.
    return output_path

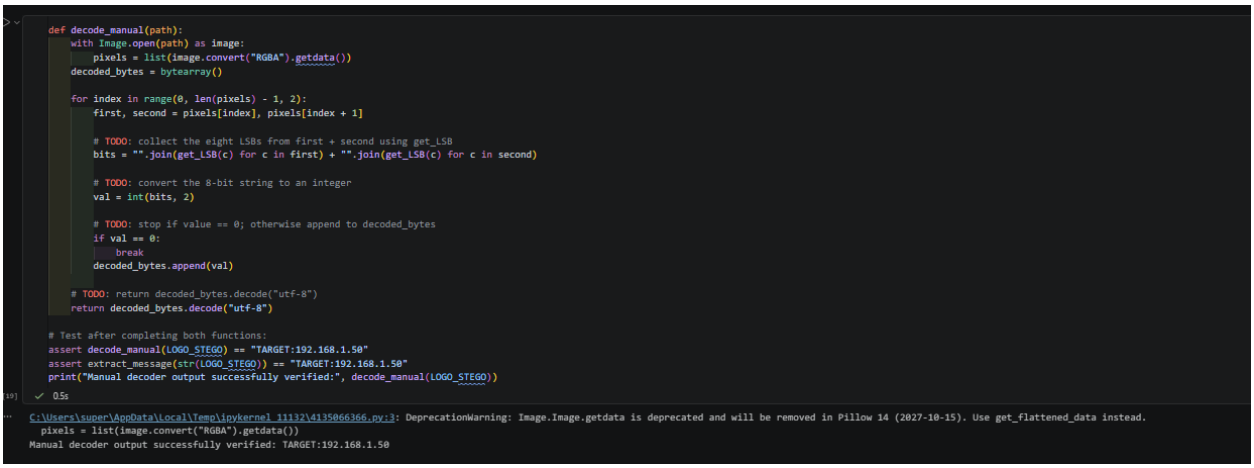
# After implementing, uncomment and run:
LOGO_STEGO = encode_checked(LOGO, "TARGET:192.168.1.50", OUTPUT_DIR / "company_logo_stego.png")
print("Company logo stego created at:", LOGO_STEGO)
```

[18] ✓ 0.4s

```
Company logo stego created at: Lab7_outputs\company_logo_stego.png
```

This confirms that the validated encoder successfully checked the payload capacity, embedded the target string, and generated the steganographic logo file at the designated output path.

Task 3C – Build a Decoder



```
def decode_manual(path):
    with Image.open(path) as image:
        pixels = list(image.convert("RGBA").getdata())
    decoded_bytes = bytearray()

    for index in range(0, len(pixels) - 1, 2):
        first, second = pixels[index], pixels[index + 1]

        # TODO: collect the eight LSBs from first + second using get_LSB
        bits = "".join(get_LSB(c) for c in first) + "".join(get_LSB(c) for c in second)

        # TODO: convert the 8-bit string to an integer
        val = int(bits, 2)

        # TODO: stop if value == 0; otherwise append to decoded_bytes
        if val == 0:
            break
        decoded_bytes.append(val)

    # TODO: return decoded_bytes.decode("utf-8")
    return decoded_bytes.decode("utf-8")

# Test after completing both functions:
assert decode_manual(LOGO_STEGO) == "TARGET:192.168.1.50"
assert extract_message(str(LOGO_STEGO)) == "TARGET:192.168.1.50"
print("Manual decoder output successfully verified:", decode_manual(LOGO_STEGO))
```

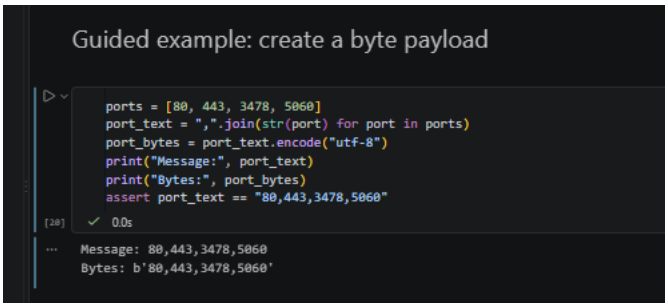
[19] ✓ 0.5s

```
C:\Users\super\AppData\Local\Temp\ipykernel_11132\4135066366.py:3: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattened_data instead.
  pixels = list(image.convert("RGBA").getdata())
Manual decoder output successfully verified: TARGET:192.168.1.50
```

This verifies that the custom manual decoder successfully extracted and decoded the hidden string, passing the assertion check with `TARGET:192.168.1.50`.

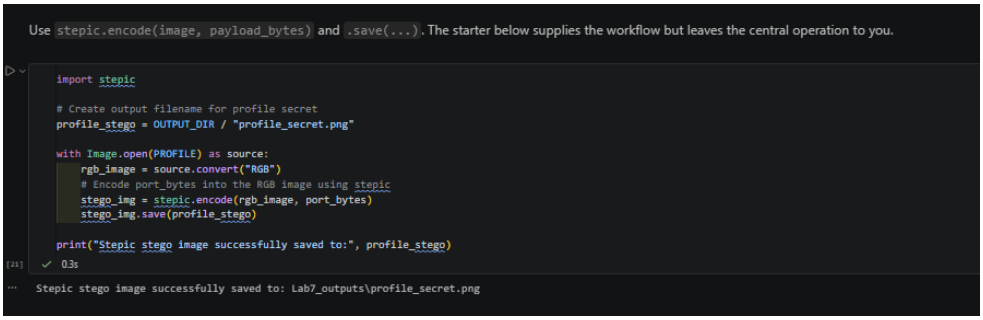
Braeden Venne

Part 4 – Stepic: Second Implementation



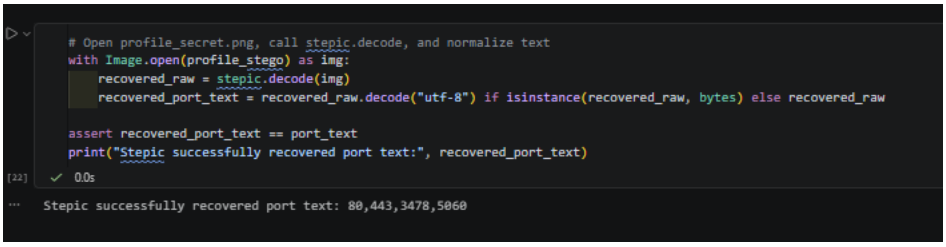
This formats the port list into a UTF-8 byte payload (`b'80,443,3478,5060`) to prepare it for steganographic embedding.

Task 4A – Encode and Save



This encodes the port byte payload into the RGB version of the profile image using `stepic.encode` and saves the resulting file to the output directory.

Task 4B – Decode and Verify



Analysis: Why is it important to verify extraction rather than assuming a successful save means the payload survived?

It is important because a successful save only indicates that the file was written to the disk, not whether the hidden payload survived. Format changes, channel conversions, or compression can silently corrupt or destroy LSB bits without throwing an error, making extraction verification essential.

Braeden Venne

Part 5 – File Integrity, Visual, and Pixel Analysis



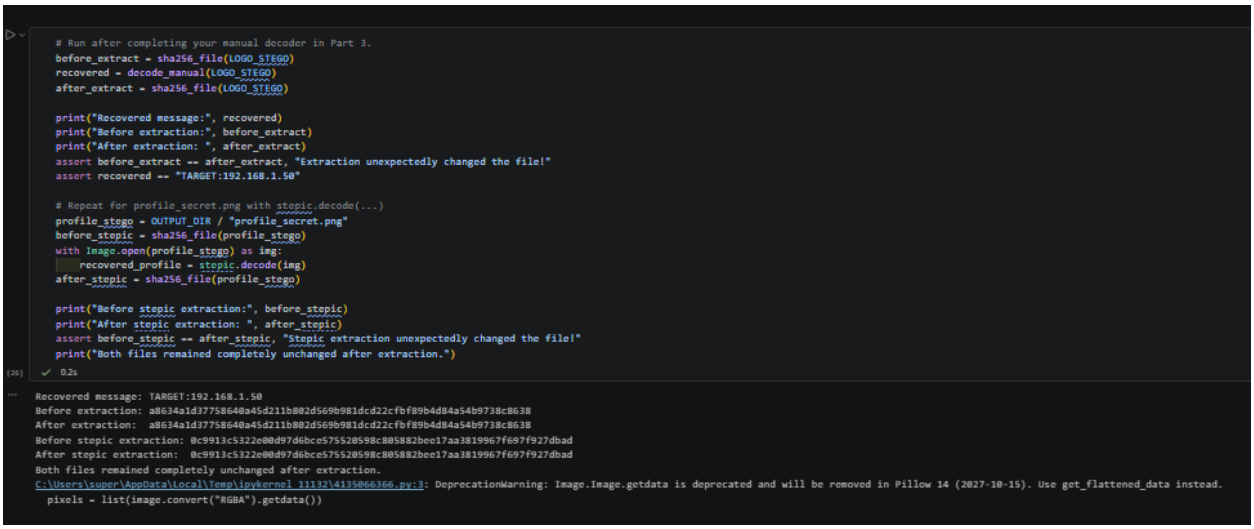
```
def compare_images(original_path, stego_path):
    with Image.open(original_path) as original, Image.open(stego_path) as stego:
        a = list(original.convert("RGBA").getdata())
        b = list(stego.convert("RGBA").getdata())
        if original.size != stego.size:
            raise ValueError("Image dimensions differ")
    changed_pixels = sum(p != q for p, q in zip(a, b))
    changed_channels = sum(x != y for p, q in zip(a, b) for x, y in zip(p, q))
    return {
        "original_bytes": Path(original_path).stat().st_size,
        "stego_bytes": Path(stego_path).stat().st_size,
        "original_sha256": sha256_file(original_path),
        "stego_sha256": sha256_file(stego_path),
        "same_file_bytes": sha256_file(original_path) == sha256_file(stego_path),
        "changed_pixels": changed_pixels,
        "changed_channels": changed_channels,
        "total_pixels": len(a),
    }

print("Reference test:", compare_images(PYTHON_IMG, TEST_STEGO))
```

```
✓ 0.0s                                                
```

Braeden Venne

Task 5B – Verify Extraction Did Not Modify the Stego-Image



```
# Run after completing your manual decoder in Part 3.
before_extract = sha256_file(LOGO_STEGO)
recovered = decode_manual(LOGO_STEGO)
after_extract = sha256_file(LOGO_STEGO)

print("Recovered message:", recovered)
print("Before extraction:", before_extract)
print("After extraction: ", after_extract)
assert before_extract == after_extract, "Extraction unexpectedly changed the file!"
assert recovered == "TARGET:192.168.1.50"

# Repeat for profile_secret.png with stepic.decode(...)
profile_stego = OUTPUT_DIR / "profile_secret.png"
before_stepic = sha256_file(profile_stego)
with Image.open(profile_stego) as img:
    recovered_profile = stepic.decode(img)
after_stepic = sha256_file(profile_stego)

print("Before stepic extraction:", before_stepic)
print("After stepic extraction: ", after_stepic)
assert before_stepic == after_stepic, "Stepic extraction unexpectedly changed the file!"
print("Both files remained completely unchanged after extraction.")
```

```
[26] ✓ 0.2s

Recovered message: TARGET:192.168.1.50
Before extraction: a8634a1d37758640a45d211b802d569b981dcd22cfbf89b4d84a54b9738c8638
After extraction:  a8634a1d37758640a45d211b802d569b981dcd22cfbf89b4d84a54b9738c8638
Before stepic extraction: 0c9913c5322e00d97d6bce57552059 8c805882bee17aa3819967f697f927dbad
After stepic extraction:  0c9913c5322e00d97d6bce57552059 8c805882bee17aa3819967f697f927dbad
Both files remained completely unchanged after extraction.
C:\Users\super\AppData\Local\Temp\ipykernel_11132\4135066366.py:3: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14 (2027-10-15). Use get_flattened_data instead.
  pixels = list(image.convert("RGBA").getdata())
```

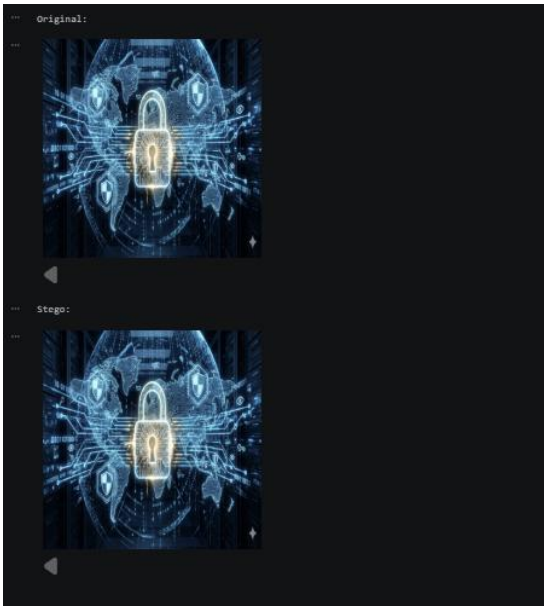
What do these hash results demonstrate?

The SHA-256 hashes before and after extraction are completely identical, proving that the decoding process is strictly read-only. It extracts the hidden payload without modifying or writing back to the file bytes, which is crucial for maintaining forensic integrity and ensuring that evidence is not contaminated or altered during investigation.

Braeden Venne

Task 5C – View Original and Modified Images





Observation: A visual comparison of both the original and stego images reveals no perceptible difference, confirming that modifying 0.0037% and 0.0040% of the pixel channels leaves the visual appearance completely intact to the human eye.

Braeden Venne

Part 6 – Independent Security Analysis and Reflection

Least-significant-bit (LSB) steganography hides data within an image's pixels without changing or altering the image's appearance. This is different from encryption because encryption scrambles content while leaving its existence obvious whereas steganography conceals the payload entirely. Steganography, however, offers no privacy if someone knows the extraction method, which allows plaintext strings to be recovered directly.

In this lab, I was able to verify that both payloads were correctly recovered, my manual decoder outputted `TARGET:192.168.1.50` and Stepic recovered `80,443,3478,5060`, both matching the original strings exactly. Image capacity scales directly with image dimensions, channels, and terminator/length encoding (my 100x100 RGBA test yielded 4999 bytes), making it dependent on these factors. Lossless formats like PNG are more appropriate for the LSB approach in this lab because with lossless formats the image data is preserved whereas lossy compression formats like JPEG alter pixel values and destroys hidden LSB data.

My SHA-256 results in this lab showcased that embedding payload data changed the exact file bytes (`same_file_bytes: False`), modifying only 0.0037% of the logo and 0.0040% of the profile. The hashes calculated before and after extraction matched perfectly, proving that the decoding process is strictly read-only and did not alter any data, meaning that forensic integrity was maintained. Additionally, different hashes do not prove steganography, they only prove that the file bytes differ, as any minor change such as metadata editing also changes the hash.

Two limitations of LSB steganography are that it lacks encryption and that it can break easily under file conversion. In an investigation two defensive steps that can be used in an authorized forensic review are Chi-Square statistical analysis to catch unnatural LSB bit distributions and file structure auditing to check for unexpected channel edits or alpha anomalies.

Finally, the TARGET IP label and port list are synthetic test values for this lab, not evidence of actual network scanning or exfiltration.